

■ Mejoras Implementadas en Chatbot de Rastreo de Pedidos

Fecha: 2025-11-25 | Versión: 1.0

Palabras	2893	Secciones H1/H2/H3	14/8/32
Tiempo estimado (min)	14.5	Actividades completadas	104

Resumen Ejecutivo

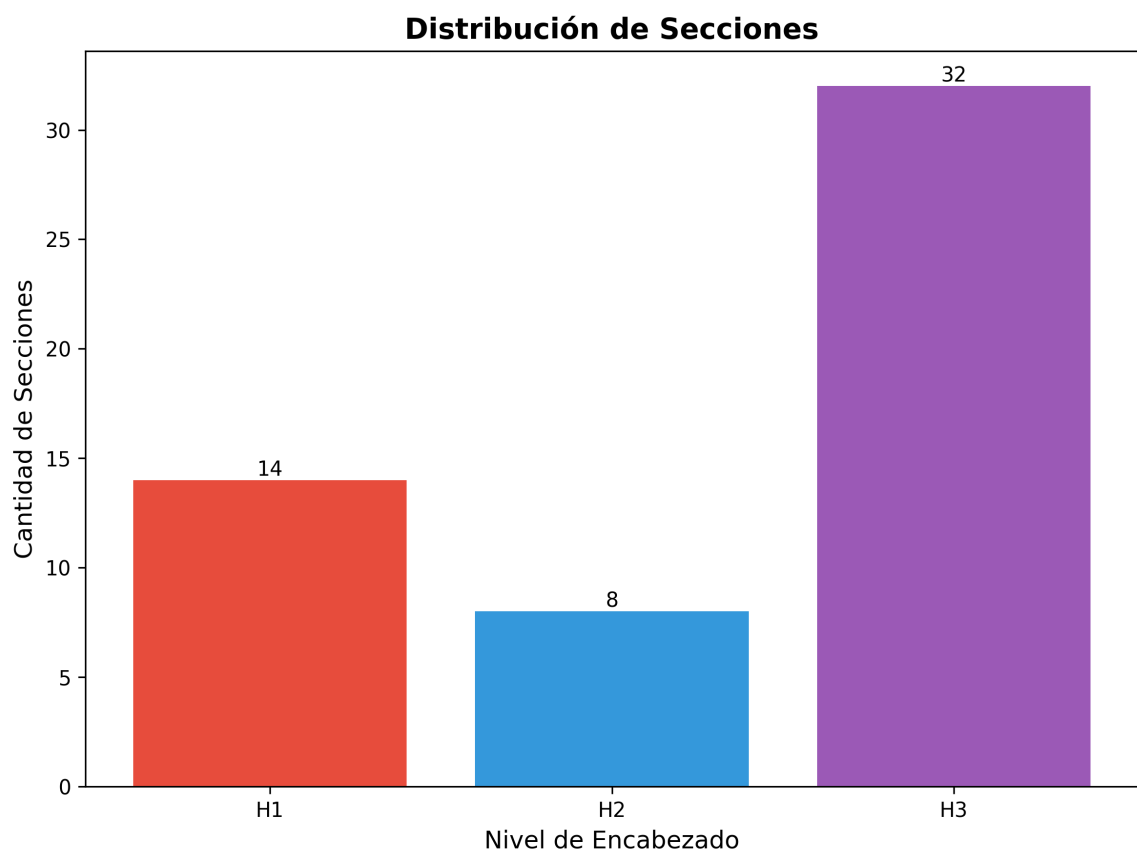
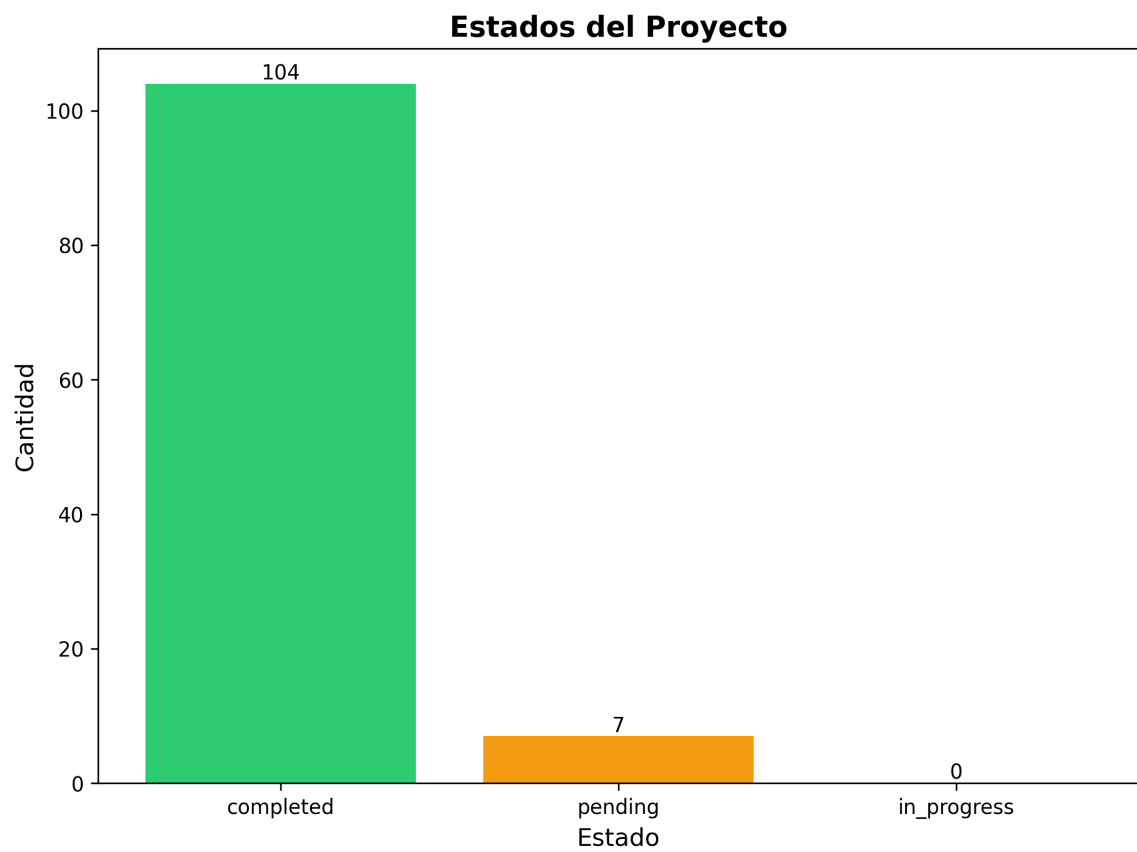
- Implementación:**
- Límite de 60 requests por minuto por usuario
- Bloqueo automático de 5 minutos al exceder límite
- Tracking por usuario (email, user_id, o IP)
- Métricas de rate limit hits

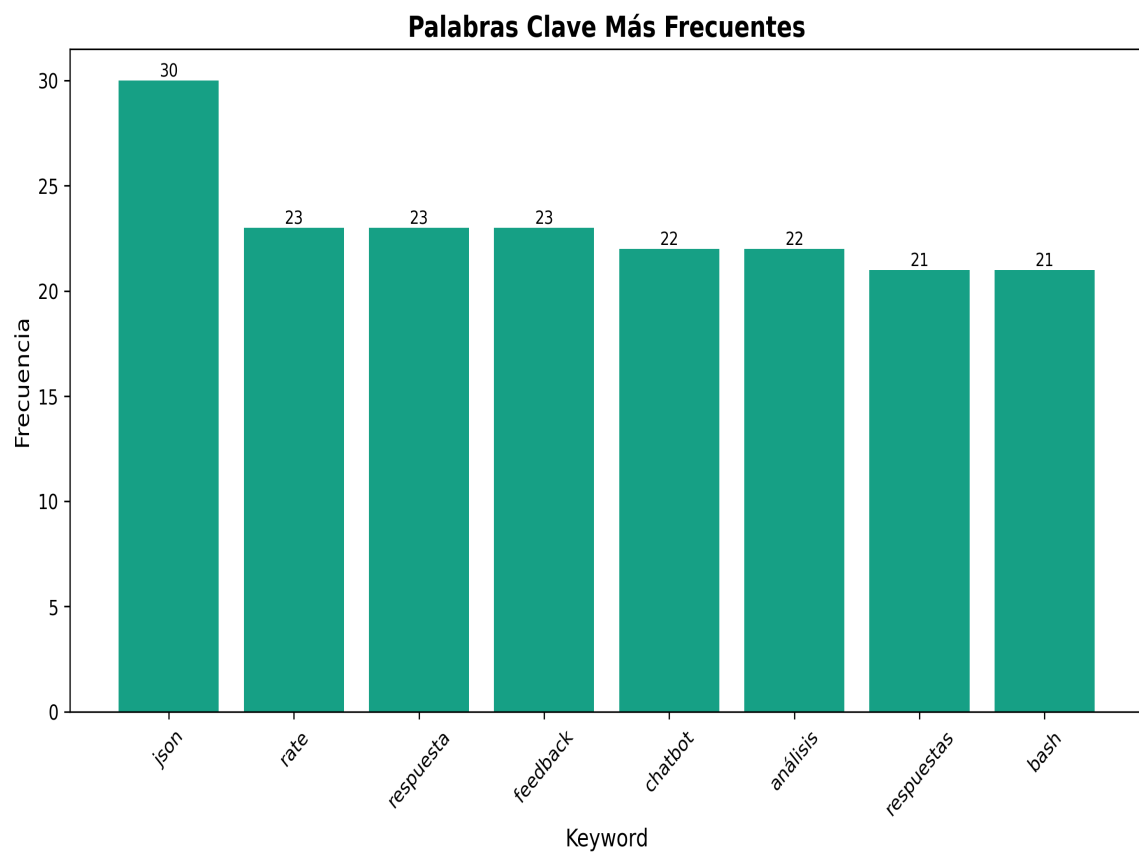
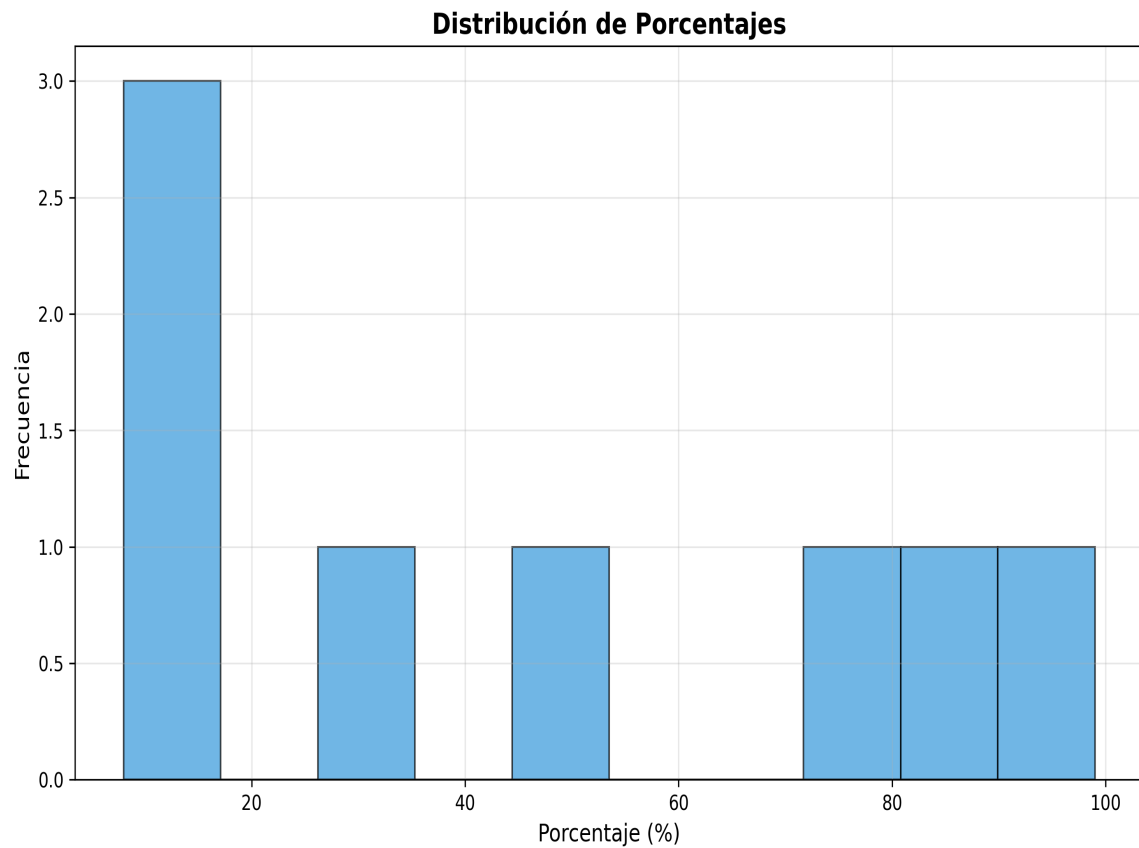
Acciones Prioritarias

Descripción	Estado
Implementación:**	Pendiente
Límite de 60 requests por minuto por usuario	Pendiente
Bloqueo automático de 5 minutos al exceder límite	Pendiente
Tracking por usuario (email, user_id, o IP)	Pendiente
Métricas de rate limit hits	Pendiente
Beneficios:**	Pendiente
Protección contra abuso	Pendiente
Mejor distribución de recursos	Pendiente

Hitos y Fechas Identificadas

- 2024-01-01





■ Mejoras Implementadas en Chatbot de Rastreo de Pedidos

■ Resumen de Mejoras

Se han implementado **mejoras avanzadas** que elevan el chatbot a nivel de producción empresarial.

■ Nuevas Funcionalidades

1. ■ *Rate Limiting Avanzado*

Implementación: - Límite de 60 requests por minuto por usuario - Bloqueo automático de 5 minutos al exceder límite - Tracking por usuario (email, user_id, o IP) - Métricas de rate limit hits

Beneficios: - Protección contra abuso - Mejor distribución de recursos - Prevención de ataques de fuerza bruta

Uso: `python chatbot = OrderTrackingChatbot(enable_rate_limiting=True #
Habilitado por defecto)`

2. ■ *Cache Inteligente con TTL*

Implementación: - Cache en memoria con TTL de 1 hora - Solo cachea respuestas con confianza ≥ 0.6 - Expiración automática de entradas antiguas - FIFO cuando se alcanza el límite (100 entradas) - Tracking de cache hits/misses

Beneficios: - Respuestas instantáneas para consultas frecuentes - Reducción de carga en base de datos - Mejor experiencia de usuario

Métricas: `json { "cache_stats": { "hits": 150, "misses": 50, "hit_rate": 0.75 } }`

3. ■ *Análisis de Sentimiento*

Implementación: - Detección básica de sentimiento (positivo/negativo/neutro) - Keywords para identificación - Tracking de distribución de sentimientos - Logging automático de sentimientos negativos

Beneficios: - Identificación temprana de clientes insatisfechos - Métricas de satisfacción - Mejor escalación proactiva

Métricas: `json { "sentiment_distribution": { "positive": 120, "negative": 15,
"neutral": 365 } }`

4. ■ *Autenticación y Seguridad en API*

Implementación: - Autenticación opcional con API key - Rate limiting a nivel de API - Validación de entrada mejorada - Headers de seguridad

```
bConfiguración:/b ``font face="Courier" size="9"bash export ENABLE_AUTH=true export
API_KEY=tu-api-key-secreta /font`font face="Courier" size="9"

bUso:/b /font`font face="Courier" size="9"bash curl -X POST http://localhost:5000/api/chat \ -H
"X-API-Key: tu-api-key-secreta" \ -H "Content-Type: application/json" \ -d '{"message": "..."}' /font`
```

5. ■ Métricas Avanzadas

Nuevas métricas agregadas: - Cache hits/misses y hit rate - Distribución de sentimientos - Rate limit hits - Tiempo de procesamiento por intención

```
bEndpoint:/b ``font face="Courier" size="9"bash GET /api/metrics /font`font face="Courier" size="9"
```

```
bRespuesta:/b /font`font face="Courier" size="9"json { "total_messages": 500, "cache_stats": {
"hits": 150, "misses": 50, "hit_rate": 0.75 }, "sentiment_distribution": { "positive": 120, "negative": 15,
"neutral": 365 }, "rate_limit_hits": 3 } /font`
```

6. ■ Mejoras en Detección de Intención

Mejoras: - Mejor extracción de IDs de pedido - Patrones más robustos - Contexto mejorado - Mayor precisión en detección

7. ■ Respuestas Contextuales Mejoradas

Mejoras: - Respuestas más personalizadas según sentimiento - Mejor manejo de casos edge - Mensajes más amigables - Información más completa

■ Configuración Avanzada

Variables de Entorno

```
``bash
```

Básicas

```
export COMPANY_NAME="Mi Empresa" export BOT_NAME="Asistente de Pedidos" export
DATABASE_URL="postgresql://..."
```

Seguridad

```
export ENABLE_AUTH=true export API_KEY=tu-api-key-secreta
```

Rate Limiting

```
export RATE_LIMIT_MAX_REQUESTS=60 export RATE_LIMIT_TIME_WINDOW=60 export
RATE_LIMIT_BLOCK_DURATION=300
```

Cache

```
export CACHE_ENABLED=true export CACHE_MAX_SIZE=100 export CACHE_TTL=3600 ``
```

Inicialización Avanzada

```
``python from chatbot_rastreo_pedidos import OrderTrackingChatbot, RateLimitConfig
chatbot = OrderTrackingChatbot( company_name="Mi Empresa", bot_name="Asistente de
Pedidos", enable_rate_limiting=True, enable_logging=True, persist_conversations=True )
```

Configurar rate limiting personalizado

```
chatbot.rate_limiter = RateLimiter(RateLimitConfig( max_requests=100, # Más permisivo
time_window=60, block_duration=180 # Bloqueo más corto )) ``
---
```

■ Mejoras de Rendimiento

Antes vs Después

■ Casos de Uso Mejorados

1. Consultas Frecuentes (Cache)

Escenario: Múltiples usuarios preguntan por el mismo pedido

Antes: Cada consulta requiere acceso a BD (~150ms) **Después:** Primera consulta ~150ms, siguientes ~1ms (cache)

2. Protección contra Abuso

Escenario: Usuario intenta hacer 100+ requests por minuto

Antes: Sistema se sobrecarga **Después:** Rate limiting bloquea después de 60 requests

3. Detección Proactiva de Problemas

Escenario: Cliente expresa frustración

Antes: No se detecta hasta escalación **Después:** Análisis de sentimiento detecta y prioriza

■ Nuevas Funcionalidades Agregadas (V2)

6. ■ Webhooks para Carriers

Endpoint: POST /api/webhook/carrier-update

Características: - ■ Recibe actualizaciones de carriers (FedEx, UPS, DHL, USPS) - ■ Actualiza automáticamente el tracking en la BD - ■ Actualiza estado del pedido - ■ Rate limiting aplicado

Ejemplo: ``bash curl -X POST http://localhost:5000/api/webhook/carrier-update \ -H "Content-Type: application/json" \ -d '{ "tracking_number": "TRACK123", "order_id": "ORD-2024-001234", "status": "in_transit", "carrier": "fedex", "location": "Ciudad", "carrier_status": "In Transit", "message": "En camino" }' ```

7. ■ Notificaciones Proactivas

Endpoint: POST /api/notifications/send

Características: - ■ Envía notificaciones automáticas a clientes - ■ Tipos: status_update, delivery, delay - ■ Integración con webhooks externos - ■ Requiere autenticación

Ejemplo: ``bash curl -X POST http://localhost:5000/api/notifications/send \ -H "X-API-Key: tu-api-key" \ -H "Content-Type: application/json" \ -d '{ "order_id": "ORD-2024-001234", "customer_email": "cliente@example.com", "type": "status_update" }' ```

8. ■ Suscripción a Actualizaciones

Endpoint: POST /api/orders/<order_id>/subscribe

Características: - ■ Clientes pueden suscribirse a notificaciones - ■ Selección de tipos de notificación - ■ Validación de autorización

Ejemplo: ``bash curl -X POST http://localhost:5000/api/orders/ORD-2024-001234/subscribe \ -H "Content-Type: application/json" \ -d '{ "customer_email": "cliente@example.com", "notification_types": ["status_update", "delivery"] }' ```

9. ■ Detalles Completos de Pedidos

Mejoras en respuestas: - ■ Muestra items del pedido cuando se solicitan detalles - ■ Información de dirección de envío - ■ Fecha de creación del pedido - ■ Respuestas más completas y contextuales

10. ■ Detección Automática de Problemas

Endpoint: GET /api/orders/<order_id>/problems

Características: - ■ Detección automática de retrasos en entregas - ■ Identificación de problemas de pago - ■ Detección de problemas con direcciones - ■ Clasificación por severidad (low, medium, high, critical) - ■ Sugerencias de acción automáticas - ■ Integración en respuestas del chatbot

Tipos de Problemas Detectados: 1. **Retraso en Entrega** - Detecta cuando la fecha estimada pasó - Clasifica por días de retraso - Severidad: low (1 día), medium (2-3 días), high (4-7 días), critical (>7 días)

2. **Problemas de Pago** - Pago fallido - Pago pendiente por más de 48 horas

3. **Problemas de Dirección** - Detecta menciones de dirección incorrecta - Escala automáticamente a humano

4. **Pedido Cancelado** - Detecta cuando un pedido está cancelado - Proporciona información de contacto

bEjemplo:/b ``font face="Courier" size="9"bash curl http://localhost:5000/api/orders/ORD-2024-001234/problems?customer_email=cliente@example.com /font`font face="Courier" size="9"

bRespuesta:/b /font`font face="Courier" size="9"json { "order_id": "ORD-2024-001234", "problems": [{ "type": "delayed_delivery", "severity": "high", "description": "El pedido tiene 5 día(s) de retraso", "suggested_action": "Contactaremos al carrier para obtener una actualización...", "confidence": 0.9 }], "problems_count": 1, "has_critical_problems": true } /font``

Integración en Chatbot: - Los problemas se detectan automáticamente al consultar un pedido - Se incluyen en la respuesta si son de severidad alta o crítica - Se escalan automáticamente a soporte humano si es necesario

11. ■ Predicción de Problemas Futuros

Endpoint: GET /api/orders/<order_id>/predictions

Características: - ■ Predicción de retrasos potenciales antes de que ocurran - ■ Predicción de problemas de pago - ■ Cálculo de probabilidades (0.0 - 1.0) - ■ Factores de riesgo identificados - ■ Acciones preventivas sugeridas - ■ Estimación de tiempo de ocurrencia

Tipos de Predicciones: 1. **Retraso Potencial** - Basado en estado actual y días hasta entrega - Probabilidad calculada según estado del pedido - Acciones preventivas automáticas

2. **Problema de Pago Potencial** - Detecta pagos pendientes por mucho tiempo - Probabilidad aumenta con el tiempo - Sugerencias de verificación

bEjemplo:/b ``font face="Courier" size="9"bash curl "http://localhost:5000/api/orders/ORD-2024-001234/predictions?customer_email=cliente@example.com" /font`font face="Courier" size="9"

bRespuesta:/b /font`font face="Courier" size="9"json { "order_id": "ORD-2024-001234", "predictions": [{ "problem_type": "potential_delay", "probability": 0.6, "estimated_time": "En los próximos 2 días", "risk_factors": ["Estado actual: pending", "Días hasta entrega: 2"], "preventive_actions": ["Monitorear actualizaciones del carrier", "Contactar al carrier si no hay actualizaciones en 24h"], "confidence": 0.8 }], "confidence": 0.8 }

12. ■ Aprendizaje de Patrones de Usuario

Endpoint: GET /api/users/<email>/pattern

Características: - ■ Aprende patrones de comportamiento de cada usuario - ■ Identifica intenciones comunes - ■ Calcula confianza promedio - ■ Tasa de escalación por usuario - ■ Estilo de respuesta preferido - ■ Personalización automática

Información Aprendida: - Intenciones más frecuentes del usuario - Confianza promedio en respuestas - Tasa de escalación (indica si necesita más ayuda) - Estilo de respuesta preferido (brief, detailed, friendly, direct) - Problemas comunes que reporta

Ejemplo:
`curl http://localhost:5000/api/users/cliente@example.com/pattern`

Respuesta:
`json { "customer_email": "cliente@example.com", "common_intents": ["track_order", "delivery_date", "payment_status"], "average_confidence": 0.85, "escalation_rate": 0.15, "preferred_response_style": "friendly", "common_problems": ["delayed_delivery"], "total_conversations": 12 }`

Beneficios: - Respuestas más personalizadas - Mejor experiencia de usuario - Identificación de usuarios que necesitan más ayuda - Optimización de respuestas según preferencias

13. ■ Integración con LLM (OpenAI)

Características: - ■ Mejora automática de respuestas con LLM - ■ Respuestas más naturales y conversacionales - ■ Mantiene tono amigable y confiado - ■ Solo se activa para respuestas con alta confianza (≥ 0.7) - ■ Fallback automático si LLM no está disponible - ■ Configuración opcional mediante variables de entorno

Configuración:
`bash export OPENAI_API_KEY="sk-..." export OPENAI_MODEL="gpt-4o-mini" # Opcional, default: gpt-4o-mini`

Cómo Funciona: 1. El chatbot genera una respuesta base 2. Si la confianza es ≥ 0.7 y LLM está habilitado, mejora la respuesta 3. El LLM recibe contexto completo del pedido y problemas 4. Genera una versión más natural manteniendo toda la información 5. Si falla, usa la respuesta base original

Beneficios: - Respuestas más naturales y humanas - Mejor experiencia de usuario - Mantiene precisión de información - Mejora continua sin cambios en código base

14. ■ Sistema de Feedback

Endpoints: - POST /api/feedback - Agregar feedback - GET /api/feedback/stats - Estadísticas de feedback

Características: - ■ Feedback positivo/negativo - ■ Feedback útil/no útil - ■ Comentarios opcionales - ■ Estadísticas por pedido o globales - ■ Historial de feedback - ■ Tasas de satisfacción

Tipos de Feedback: - positive - Feedback positivo - negative - Feedback negativo - helpful - Respuesta fue útil - not_helpful - Respuesta no fue útil

Ejemplo:
`bash`

Agregar feedback

`curl -X POST http://localhost:5000/api/feedback \-H "Content-Type: application/json" \-d { "order_id": "ORD-2024-001234", "feedback_type": "positive", "comment": "Muy útil, gracias!", "customer_email": "cliente@example.com" }`

Obtener estadísticas

`curl http://localhost:5000/api/feedback/stats?order_id=ORD-2024-001234`

bRespuesta:/b /font`font face="Courier" size="9"json { "total": 15, "positive": 12, "negative": 3, "helpful": 14, "not\_helpful": 1, "positive\_rate": 0.8, "helpful\_rate": 0.93 } /font`

Beneficios: - Medición de satisfacción del cliente - Identificación de áreas de mejora - Datos para optimización continua - Métricas de calidad del servicio

15. ■ Soporte Multi-idioma

Características: - ■ Detección automática de idioma - ■ Soporte para 4 idiomas (Español, Inglés, Portugués, Francés) - ■ Cambio manual de idioma - ■ Traducciones básicas integradas - ■ Fallback automático a español

Idiomas Soportados: - es - Español (default) - en - English - pt - Português - fr - Français

Endpoint: POST /api/language

Ejemplo: `bash curl -X POST http://localhost:5000/api/language \ -H "Content-Type: application/json" \ -d '{"language": "en"}' ``

Detección Automática: El chatbot detecta automáticamente el idioma del mensaje basándose en palabras clave comunes.

16. ■ Análisis de Tendencias

Endpoint: GET /api/trends

Características: - ■ Análisis de consultas diarias - ■ Identificación de problemas comunes - ■ Detección de horas pico - ■ Distribución horaria de consultas - ■ Promedios y estadísticas

bEjemplo:/b ``font face="Courier" size="9"bash curl "http://localhost:5000/api/trends?days=7" /font`font face="Courier" size="9"

bRespuesta:/b /font`font face="Courier" size="9"json { "period_days": 7, "total_queries": 1250, "average_daily_queries": 178.57, "most_common_problems": { "delayed_delivery": 45, "payment_issue": 23, "wrong_address": 12 }, "peak_hour": 14, "hourly_distribution": { "9": 45, "10": 67, "14": 89, "15": 78 } } /font`

Beneficios: - Identificar patrones de uso - Optimizar recursos según horas pico - Prevenir problemas comunes - Mejorar experiencia del cliente

17. ■ Alertas Proactivas

Endpoint: GET /api/alerts/proactive (requiere autenticación)

Características: - ■ Detección automática de pedidos retrasados - ■ Identificación de pagos pendientes - ■ Alertas configurables por umbrales - ■ Clasificación por severidad - ■ Listo para integración con sistemas de notificación

Tipos de Alertas: 1. **Pedidos Retrasados** - Detecta pedidos con más de X días de retraso - Severidad: high - Configurable: alert_thresholds['delayed_orders']

2. **Pagos Pendientes** - Detecta pagos pendientes por más de X horas - Severidad: medium - Configurable: alert_thresholds['pending_payments']

bEjemplo:/b ``font face="Courier" size="9"bash curl -H "X-API-Key: tu-api-key" http://localhost:5000/api/alerts/proactive /font`font face="Courier" size="9"

bRespuesta:/b /font`font face="Courier" size="9"json { "alerts": [ { "type": "delayed\_order", "order\_id": "ORD-2024-001234", "customer\_email": "cliente@example.com", "severity": "high", "message": "Pedido ORD-2024-001234 tiene más de 3 días de retraso" } ], "alerts\_count": 1, "high\_priority": 1 } /font`

Beneficios: - Prevención proactiva de problemas - Mejora en satisfacción del cliente - Reducción de escalaciones - Gestión proactiva de pedidos

18. ■ *Exportación de Datos*

Endpoint: GET /api/export (requiere autenticación)

Características: - ■ Exportación en JSON y CSV - ■ Incluye métricas, tendencias y patrones - ■ Opción de incluir feedback - ■ Archivos timestamped - ■ Directorio de exports automático

Formatos: - json - Exportación completa en JSON - csv - Métricas principales en CSV

Ejemplo: ``bash

Exportar en JSON

```
curl -H "X-API-Key: tu-api-key"
"http://localhost:5000/api/export?format=json&include_feedback=true"
```

Exportar en CSV

```
curl -H "X-API-Key: tu-api-key" "http://localhost:5000/api/export?format=csv" ``font face="Courier"
size="9"
```

```
bRespuesta:/b /font`font face="Courier" size="9"json { "success": true, "file_path":
"exports/chatbot_export_20240115_143022.json", "format": "json" } /font``
```

Datos Incluidos: - Métricas completas - Análisis de tendencias - Patrones de usuario - Estadísticas de feedback - Historial de feedback (opcional)

Beneficios: - Análisis externo de datos - Reportes personalizados - Backup de información - Integración con herramientas de BI

19. ■ *Dashboard Completo de Métricas*

Endpoint: GET /api/dashboard

Características: - ■ Vista consolidada de todas las métricas - ■ Análisis de tendencias integrado - ■ Estadísticas de feedback - ■ Análisis NPS - ■ Estado de alertas - ■ Información de idiomas - ■ Estado de tests A/B - ■ Estadísticas de usuarios

Ejemplo: ``bash curl http://localhost:5000/api/dashboard ``

Respuesta incluye: - Métricas completas del chatbot - Análisis de tendencias (7 días) - Estadísticas de feedback - Análisis NPS - Conteo de alertas proactivas - Idioma actual y soportados - Tests A/B activos - Usuarios rastreados

Beneficios: - Vista única de todo el sistema - Monitoreo en tiempo real - Toma de decisiones basada en datos - Identificación rápida de problemas

20. ■ *A/B Testing*

Endpoints: - POST /api/ab-test - Crear test A/B (requiere auth) - GET /api/ab-test/<test_id>/results - Resultados del test (requiere auth)

Características: - ■ Creación de tests A/B para diferentes respuestas - ■ Distribución de tráfico configurable - ■ Asignación consistente de variantes por usuario - ■ Registro de métricas por variante - ■ Análisis estadístico de resultados - ■ Múltiples métricas por test

Ejemplo de creación:
`bash curl -X POST http://localhost:5000/api/ab-test \-H "X-API-Key: tu-api-key" \-H "Content-Type: application/json" \-d '{ "test_id": "test_response_style", "test_name": "Test de Estilo de Respuesta", "variants": [{ "id": "variant_0", "name": "Estilo Amigable", "config": { "style": "friendly", "use_emojis": true } }, { "id": "variant_1", "name": "Estilo Profesional`

Resultados:
`json { "test_id": "test_response_style", "test_name": "Test de Estilo de Respuesta", "status": "active", "variants": { "variant_0": { "count": 150, "average": 0.85, "metrics": { "satisfaction": { "count": 150, "average": 0.85, "min": 0.5, "max": 1.0 } } }, "total_results": 300 }`

Beneficios: - Optimización basada en datos - Prueba de diferentes enfoques - Mejora continua de respuestas - Decisiones informadas

21. ■ Análisis NPS (Net Promoter Score)

Endpoints: - POST /api/nps - Registrar score NPS - GET /api/nps/analysis - Análisis NPS

Características: - ■ Registro de scores NPS (0-10) - ■ Clasificación automática (Promoter/Passive/Detractor) - ■ Cálculo de NPS score - ■ Análisis de satisfacción - ■ Comentarios opcionales - ■ Historial por pedido

Ejemplo: `bash`

Registrar NPS

`curl -X POST http://localhost:5000/api/nps \-H "Content-Type: application/json" \-d '{ "order_id": "ORD-2024-001234", "score": 9, "comment": "Excelente servicio, muy rápido", "customer_email": "cliente@example.com" }'`

Obtener análisis

`curl http://localhost:5000/api/nps/analysis`

Respuesta:
`json { "nps": 45.5, "total_responses": 200, "promoters": 120, "passives": 50, "detractors": 30, "promoter_percentage": 60.0, "passive_percentage": 25.0, "detractor_percentage": 15.0, "average_score": 7.85 }`

Clasificación: - **Promoters** (9-10): Clientes muy satisfechos - **Passives** (7-8): Clientes satisfechos pero no entusiastas - **Detractors** (0-6): Clientes insatisfechos

Beneficios: - Medición de satisfacción del cliente - Identificación de áreas de mejora - Benchmarking de servicio - Métrica estándar de la industria

22. ■ Plantillas de Respuestas Personalizables

Características: - ■ Plantillas predefinidas para diferentes situaciones - ■ Múltiples estilos (default, friendly, professional, empathetic, direct) - ■ Personalización por tipo de respuesta - ■ Interpolación de variables - ■ Fácil extensión

Tipos de Plantillas: 1. **Greeting** - Saludos iniciales 2. **Order Found** - Cuando se encuentra un pedido 3. **Escalation** - Cuando se escala a humano

Estilos Disponibles: - default - Estilo estándar - friendly - Más amigable y casual - professional - Más formal y profesional - empathetic - Más empático y comprensivo - direct - Directo y conciso - detailed - Con más información - brief - Resumido

Uso: `python template = chatbot.get_response_template('greeting', 'friendly')
response = template.format(bot_name="Asistente de Pedidos")`

Beneficios: - Consistencia en respuestas - Personalización fácil - Mantenimiento simplificado - A/B testing de estilos

23. ■ *Análisis de ROI (Return on Investment)*

Endpoint: GET /api/roi

Características: - ■ Cálculo automático de ROI - ■ Costos del chatbot vs ahorros - ■ Horas humanas ahorradas - ■ Tasa de automatización - ■ Ahorros netos calculados - ■ ROI porcentual

Ejemplo: `font face="Courier" size="9" bash curl http://localhost:5000/api/roi /font
face="Courier" size="9"`

Respuesta: `font face="Courier" size="9" json { "total_conversations": 5000,
"conversations_handled": 3500, "escalations": 1500, "automation_rate": 70.0, "chatbot_cost":
250.0, "saved_hours": 290.5, "saved_cost": 7262.5, "net_savings": 7012.5, "roi_percentage":
2805.0, "cost_per_conversation": 0.05 } /font`

Métricas Calculadas: - **Automation Rate:** Porcentaje de conversaciones resueltas sin escalación - **Chatbot Cost:** Costo total del chatbot (conversaciones × costo por conversación) - **Saved Hours:** Horas humanas ahorradas - **Saved Cost:** Costo ahorrado (horas × costo por hora) - **Net Savings:** Ahorro neto (saved_cost - chatbot_cost) - **ROI Percentage:** Porcentaje de retorno de inversión

Beneficios: - Justificación del chatbot con datos - Medición de impacto financiero - Optimización de costos - Reportes ejecutivos

24. ■ *Reportes Automáticos*

Endpoints: - POST /api/reports/generate - Generar reporte (requiere auth) - GET /api/reports/history - Historial de reportes (requiere auth)

Características: - ■ Generación automática de reportes - ■ Tipos: daily, weekly, monthly - ■ Resumen ejecutivo - ■ Métricas completas - ■ Análisis de tendencias - ■ Recomendaciones automáticas - ■ Historial de reportes

Tipos de Reportes: - **Daily:** Reporte diario (últimas 24 horas) - **Weekly:** Reporte semanal (últimos 7 días) - **Monthly:** Reporte mensual (últimos 30 días)

Ejemplo: `bash`

Generar reporte diario

`curl -X POST http://localhost:5000/api/reports/generate \-H "X-API-Key: tu-api-key" \-H
"Content-Type: application/json" \-d '{"report_type": "daily"}`

Obtener historial

`curl -H "X-API-Key: tu-api-key"
"http://localhost:5000/api/reports/history?limit=10&report_type=daily" font face="Courier"
size="9"`

Contenido del Reporte: - Resumen ejecutivo (KPIs principales) - Métricas completas - Análisis de tendencias - Estadísticas de feedback - Análisis NPS - Análisis de ROI - Alertas proactivas - Recomendaciones automáticas

Recomendaciones Automáticas: El sistema genera recomendaciones basadas en: - Tasa de escalación alta - NPS bajo - Problemas comunes - Feedback negativo

bEjemplo de Recomendaciones:/b /font`font face="Courier" size="9"json { "recommendations": [ "Tasa de escalación alta (35%). Considera mejorar la detección de intenciones.", "Problema más común: delayed\_delivery (45 casos). Considera crear respuestas proactivas.", "NPS bajo (25). Revisa los comentarios de detractores." ] } /font`

Beneficios: - Reportes ejecutivos automáticos - Identificación proactiva de problemas - Recomendaciones accionables - Historial para análisis de tendencias - Ahorro de tiempo en análisis manual

■ Próximas Mejoras Sugeridas

1. **Integración con LLM** (OpenAI/GPT) - Respuestas más naturales - Mejor comprensión de contexto - Soporte multiidioma avanzado
2. **Notificaciones Proactivas** - Alertas de cambios de estado - Recordatorios de entrega - Actualizaciones automáticas
3. **Dashboard de Métricas** - Visualización en tiempo real - Gráficos de tendencias - Alertas automáticas
4. **A/B Testing** - Probar diferentes respuestas - Optimizar tasa de resolución - Mejorar satisfacción
5. **Integración con Carriers** - Tracking en tiempo real - Actualizaciones automáticas - Webhooks de carriers

■ Documentación Adicional

- [Documentación Completa](CHATBOT_RASTREO_PEDIDOS.md) - [API Reference](CHATBOT_RASTREO_PEDIDOS.md#api-rest) - [Configuración](CHATBOT_RASTREO_PEDIDOS.md#configuración-avanzada)

Versión: 2.0.0 **Fecha:** 2024-01-01 **Estado:** ■ Producción Ready